Practical: 06 **Date: 01-10-2026**

Bias-Variance Trade-off

Procedure:

Step 1: Define the Polynomial Regression helper function using Scikit-Learn's make pipeline to combine Polynomial Features and Linear Regression.

Step 2: Define the make data function to generate non-linear synthetic data with random noise, and generate a initial dataset with 50 points (X, y).

Step 3: Plot the 50-point dataset along with fitted polynomial regression curves for degrees 1, 3, and 5 to visualize underfitting and overfitting.

Step 4: Calculate training and validation scores for degrees 0 through 20 on the 50-point dataset using validation curve with 7-fold cross-validation, passing param name and param range as explicit keyword arguments.

Step 5: Plot the resulting training and validation curves to locate the optimal model degree for the 50-point dataset.

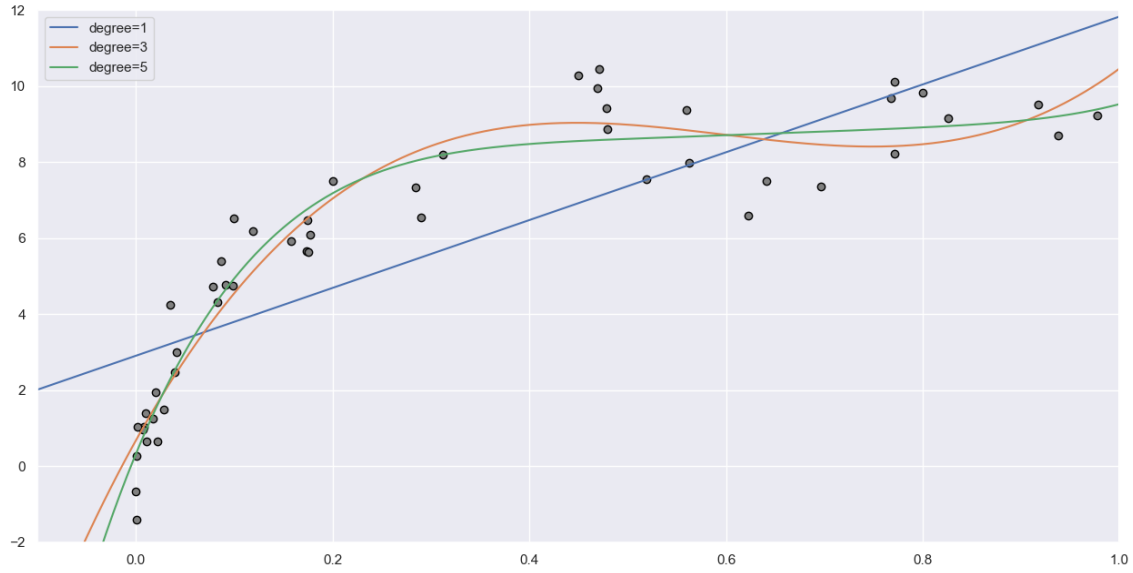
Step 6: Generate a larger dataset of 200 samples (X2, y2) using make data (200) and plot its scatter distribution.

Step 7: Compute the new validation curve for the 200-point dataset and plot it alongside the original 50-point curve (shown as dashed lines) to visualize how increasing dataset size impacts model complexity and performance.

Solution:

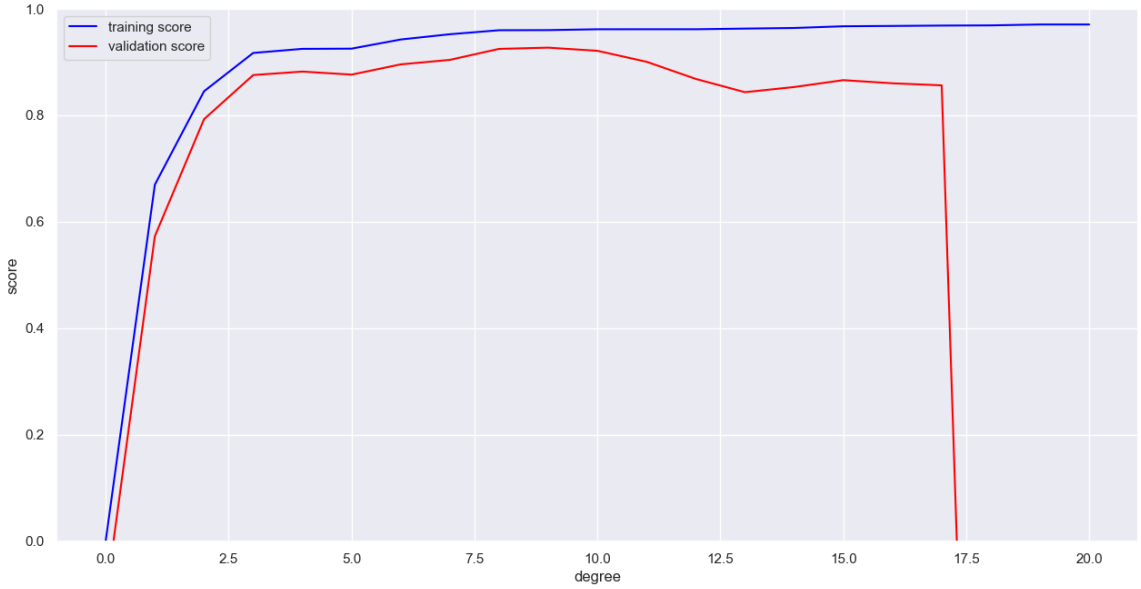
```
from sklearn.preprocessing import PolynomialFeatures
from sklearn.linear_model import LinearRegression
from sklearn.pipeline import make_pipeline
def PolynomialRegression(degree=2, **kwargs):
    return make_pipeline(PolynomialFeatures(degree),
                         LinearRegression(**kwargs))
import numpy as np
def make_data(N, err=1.0, rseed=1):
    # randomly sample the data
    rng = np.random.RandomState(rseed)
    X = rng.rand(N, 1) ** 2
    y = 10 - 1. / (X.ravel() + 0.1)
    if err > 0:
        y += err * rng.randn(N)
    return X, y
X, y = make_data(50)
```

```
import matplotlib.pyplot as plt
import seaborn; seaborn.set()  # plot formatting
X_test = np.linspace(-0.1, 1.1, 500)[:, None]
plt.scatter(X.ravel(), y, edgecolor='black', color='gray')
axis = plt.axis()
for degree in [1, 3, 5]:
    y_test = PolynomialRegression(degree).fit(X, y).predict(X_test)
    plt.plot(X_test.ravel(), y_test, label='degree={0}'.format(degree))
plt.xlim(-0.1, 1.0)
plt.ylim(-2, 12)
plt.legend(loc='best')
plt.show()
```




```
from sklearn.model_selection import validation_curve
degree = np.arange(0, 21)
train_score, val_score = validation_curve(PolynomialRegression(), X, y,
                                          param_name='polynomialfeatures__degree', param_range=degree, cv=7)

plt.plot(degree, np.median(train_score, 1), color='blue', label='training score')
plt.plot(degree, np.median(val_score, 1), color='red', label='validation score')
plt.legend(loc='best')
plt.ylim(0, 1)
plt.xlabel('degree')
plt.ylabel('score')
plt.show()
plt.figure()
```




```
X2, y2 = make_data(200)
plt.scatter(X2.ravel(), y2);
degree = np.arange(21)
train_score2, val_score2 = validation_curve(
    PolynomialRegression(), X2, y2,
    param_name='polynomialfeatures__degree',
    param_range=degree,
    cv=7
)
plt.plot(degree, np.median(train_score2, 1), color='blue', label='training score')
plt.plot(degree, np.median(val_score2, 1), color='red', label='validation score')
plt.plot(degree, np.median(train_score, 1), color='blue', alpha=0.3, linestyle='dashed')
plt.plot(degree, np.median(val_score, 1), color='red', alpha=0.3, linestyle='dashed')
plt.legend(loc='lower center')
plt.ylim(0, 1)
plt.xlabel('degree')
plt.ylabel('score')
plt.show()
```

